

Assignment 6

Submitted by: CSB24008

Complexity Analysis Java Program

1. Time Complexity

1.1 Time Complexity of Computing Course Averages

The method `getAverageScorePerCourse()` computes the average score for each course taken by the students.

The algorithm works as follows:

1. The program iterates through the list of students.
2. For every student, it iterates through the courses that the student has taken.
3. For each course, it updates the total score and count inside a HashMap.

The structure of the algorithm can be represented as:

```
for each student in students
  for each course in student.courses
    update course statistics
```

Let:

- **S** = number of students
- **C** = average number of courses per student

The outer loop runs **S times**, and the inner loop runs **C times**.

Therefore, the total number of operations becomes:

$$S \times C$$

After this, the algorithm iterates once through the course statistics map to compute the final averages. Since the number of courses is small and fixed, this step requires **O(C)** time.

Thus, the overall time complexity becomes:

$$O(S \times C)$$

Since the number of courses is relatively small and constant in this problem (Math, DSA, Java, OS, DBMS), the complexity behaves approximately like:

$$O(S)$$

in practice.

Verification Using Experimental Output

From the program outputs:

Number of Students	Course Average Time (ns)
1000	3,814,300
10000	13,296,900
100000	41,495,000

Now we compare the increase.

When the number of students increases:

$$1000 \rightarrow 10000$$

which is **10× larger**, the execution time increases from:

$$3,814,300 \rightarrow 13,296,900$$

The ratio is:

$$\frac{13,296,900}{3,814,300} \approx 3.48$$

Similarly, when the students increase:

$$10000 \rightarrow 100000$$

which is again **10× larger**, the execution time increases from:

$$13,296,900 \rightarrow 41,495,000$$

The ratio is:

$$\frac{41,495,000}{13,296,900} \approx 3.12$$

Although the time does not increase exactly 10 times due to JVM optimizations and hardware factors, the increase still follows a **roughly linear trend**, which supports the **$O(S \times C)$** complexity.

2. Time Complexity of Sorting Top N Students

The method `getTopNStudents()` sorts students based on their average scores.

The relevant code is:

```
students.stream()
    .sorted(Comparator.comparingDouble(Student::getAverageScore).reversed())
    .limit(n)
```

The most expensive operation here is **sorting**.

Sorting **S elements** using comparison-based algorithms requires:

$$O(S \log S)$$

operations.

After sorting, the `limit(n)` operation simply extracts the first **N students**, which requires **$O(N)$** time.

Since sorting dominates the computation, the overall time complexity becomes:

$$O(S \log S)$$

Verification Using Experimental Output

From the program output:

Students	Sorting Time (ns)
1000	34,351,500
10000	43,014,500
100000	77,746,300

To observe the growth:

When students increase from:

$$1000 \rightarrow 10000$$

sorting time increases from:

$$34,351,500 \rightarrow 43,014,500$$

Ratio:

$$\frac{43,014,500}{34,351,500} \approx 1.25$$

When students increase from:

$$10000 \rightarrow 100000$$

sorting time increases from:

$$43,014,500 \rightarrow 77,746,300$$

Ratio:

$$\frac{77,746,300}{43,014,500} \approx 1.81$$

The increase is **faster than linear growth**, which is consistent with the theoretical complexity of **$O(S \log S)$** .

Sorting therefore becomes the **dominant operation** as the number of students increases.

3. Space Complexity

3.1 Space Complexity of Computing Course Averages

The algorithm uses a HashMap to store statistics for each course:

course $\rightarrow$ [total score, count]

If **C** is the number of unique courses, the map stores **C entries**.

Thus, the additional space required is:

$$O(C)$$

3.2 Space Complexity of Sorting Top N Students

Sorting requires temporary memory proportional to the number of elements being sorted.

Thus, sorting **S students** requires:

$$O(S)$$

space.

The result list containing the **top N students** requires:

$$O(N)$$

space.

Therefore, the total space complexity becomes:

$$O(S + N)$$

Since **S is much larger than N**, the complexity simplifies to:

$$O(S)$$

Final Conclusion

The analysis of the algorithms and the experimental results obtained from the program execution confirm that:

- Computing course averages has a time complexity of **$O(S \times C)$** .
- Sorting students to obtain the top N students has a time complexity of **$O(S \log S)$** .
- The additional space required for the algorithms is **$O(C)$** for course averages and **$O(S)$** for sorting operations.

These results are consistent with the theoretical expectations for the algorithms used in the program.

Enter number of students: 1000

Top 5 Students:

808 - Student808 | Avg: 96.75

265 - Student265 | Avg: 96.60

514 - Student514 | Avg: 96.33

830 - Student830 | Avg: 96.00

997 - Student997 | Avg: 95.33

Course Averages:

{Java=80.37046307884856, DSA=80.39348370927318, OS=80.47420147420148, DBMS=79.05183312262959, Math=79.33048780487805}

Unique Courses:

[Java, DSA, OS, DBMS, Math]

Execution Time:

Top N Students: 34351500 ns

Course Averages: 3814300 ns

Unique Courses: 16331300 ns

Enter number of students: 10000

Top 5 Students:

9016 - Student9016 | Avg: 99.67

8450 - Student8450 | Avg: 98.40

2805 - Student2805 | Avg: 98.33

7230 - Student7230 | Avg: 98.33

8310 - Student8310 | Avg: 97.67

Course Averages:

{Java=79.6320340894849, DSA=80.16668753913588, OS=79.9346347607053, DBMS=79.91505550704753, Math=80.02897641746111}

Unique Courses:

[Java, DSA, OS, DBMS, Math]

Execution Time:

Top N Students: 43014500 ns

Course Averages: 13296900 ns

Unique Courses: 12445900 ns

Enter number of students: 100000

Top 5 Students:

39870 - Student39870 | Avg: 99.67

53904 - Student53904 | Avg: 99.67

2996 - Student2996 | Avg: 99.33

98071 - Student98071 | Avg: 99.33

68651 - Student68651 | Avg: 99.25

Course Averages:

{Java=79.94989712575597, DSA=79.9810701444482, OS=79.98735536055322, DBMS=80.02626740215393, Math=80.02089615735933}

Unique Courses:

[Java, DSA, OS, DBMS, Math]

Execution Time:

Top N Students: 77746300 ns

Course Averages: 41495000 ns

Unique Courses: 31055200 ns